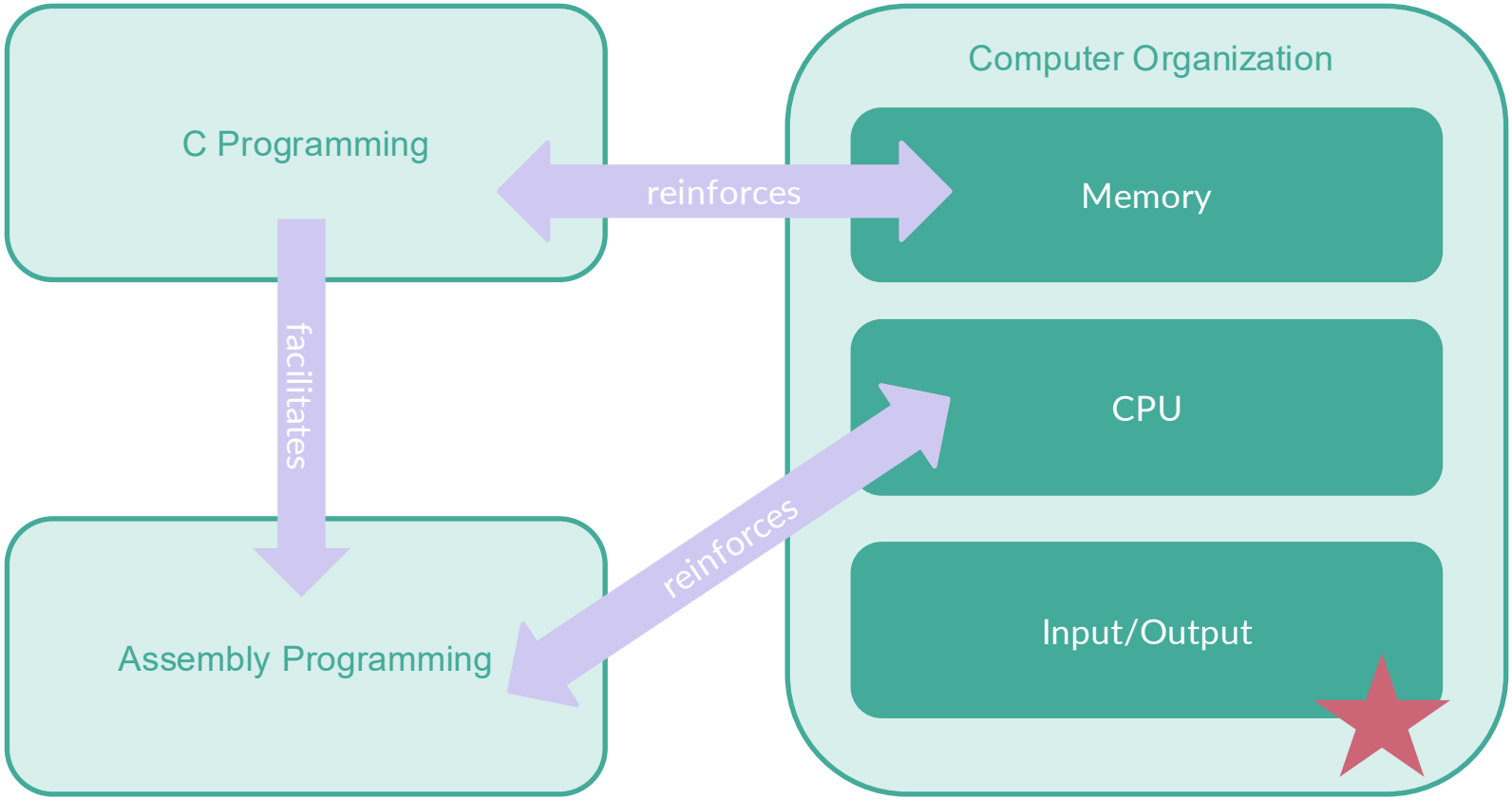
CS 255: Computer Architecture and Machine Level Programming

Prof. Martha Barker

Introduction to Computer Organization

Components of the Course



Computer Organization Describes Hardware Implementation

```c
#include <stdlib.h>
#include <stdio.h>
#include <string.h>

int main(int argc, char* argv[])
{
  int iter;
  double x, y, z;

  printf("Iterations: ");
  scanf("%d",&iter);

  /* initialize random numbers */
  srand(123456789);

  /* cnt = number of points in the
     1st quadrant of unit circle */
  int cnt = 0;
  int i = 0;
  while (i++ < iter){
    x = (double)rand() / RAND_MAX;
    y = (double)rand() / RAND_MAX;
    z = x*x + y*y;
    if (z <= 1) cnt++;
  }
  double pi=(double)cnt / iter * 4;
  printf("pi is %g\n", pi);
  return 0;
}
```



Why Does This Matter?

Understanding computer organization matters because:

- Every program you write runs on physical hardware
- Every variable in your code must be stored and moved
- Every computation requires time and resources
- Every abstraction you use maps to hardware behavior

What Is A Computer?

Discussion: What is a Computer?

What Is A Computer?

A **computer** is a programmable computing device.

What Is A Computer?

A **computer** is a programmable computing device.

It performs computations – it takes in inputs, applies a set of operations, and produces outputs.

What Is A Computer?

A **computer** is a programmable computing device.

Its behavior isn't fixed – it follows a stored sequence of instructions that can be updated.

It performs computations – it takes in inputs, applies a set of operations, and produces outputs.

There Are Many Kinds of Computers

Personal computers
- laptop, desktop

Less-traditional personal computers
- smart phone, tablet, gaming system

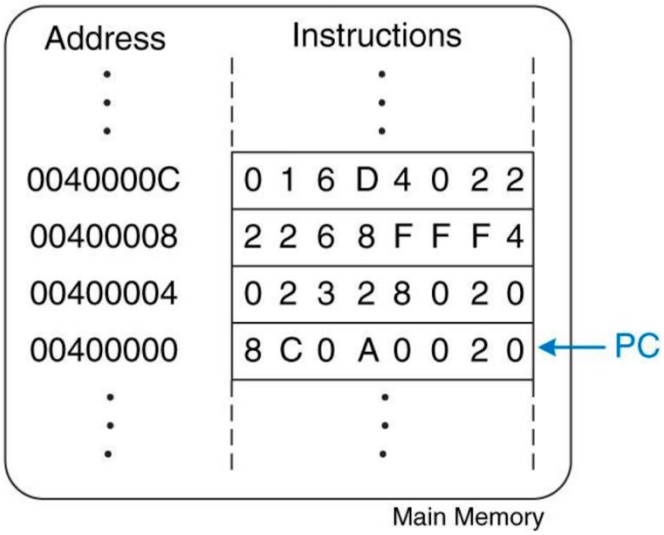
Big shared computers
- cloud computers, lab servers

Small embedded computers
- microcontrollers in cars, ATMs, smart devices

The von Neumann Architecture

The **von Neumann Architecture** is (1) a general purpose computer that uses (2) a stored-program model.

Assembly Code	Machine Code
`lw $t2, 32($0)`	0x8C0A0020
`add $s0, $s1, $s2`	0x02328020
`addi $t0, $s3, -12`	0x2268FFF4
`sub $t0, $t3, $t5`	0x016D4022



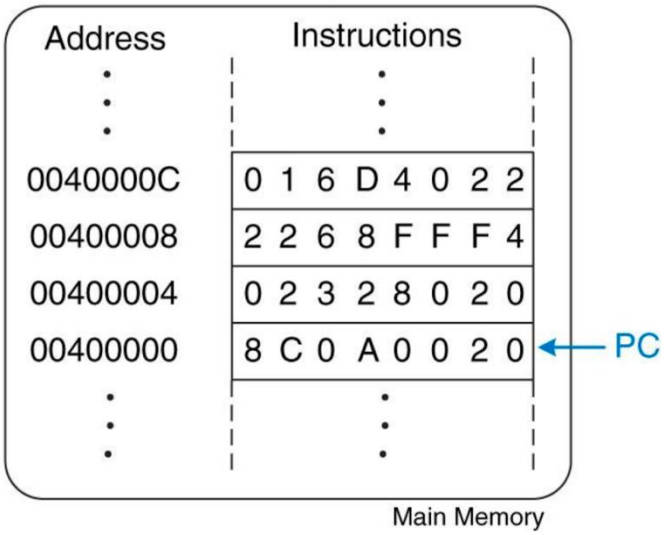
[Harris & Harris]

The von Neumann Architecture

The **von Neumann Architecture** is (1) a general purpose computer that uses (2) a stored-program model.

Program instructions are stored in memory, so running a new program is as simple as storing new instructions in memory.

Assembly Code	Machine Code
`lw $t2, 32($0)`	0x8C0A0020
`add $s0, $s1, $s2`	0x02328020
`addi $t0, $s3, -12`	0x2268FFF4
`sub $t0, $t3, $t5`	0x016D4022

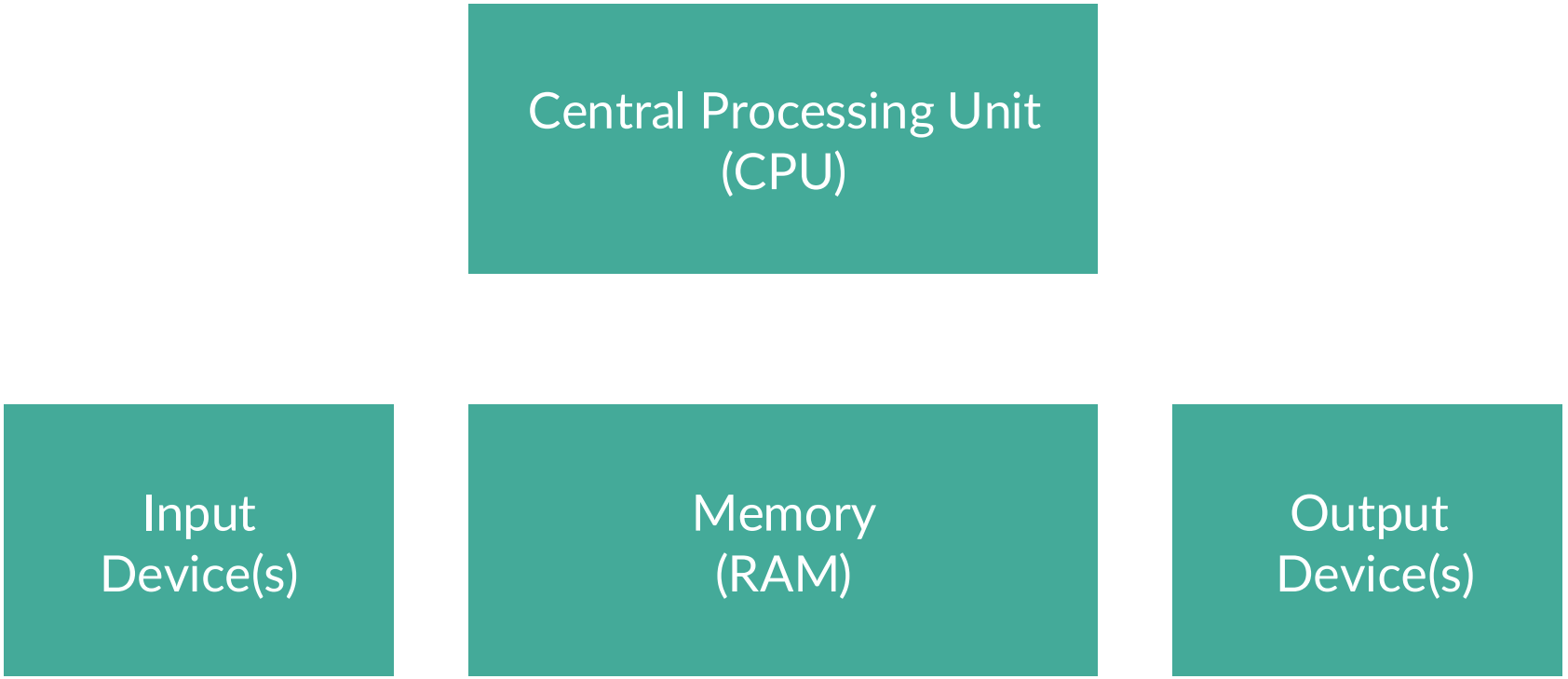


[Harris & Harris]

Logical Structure

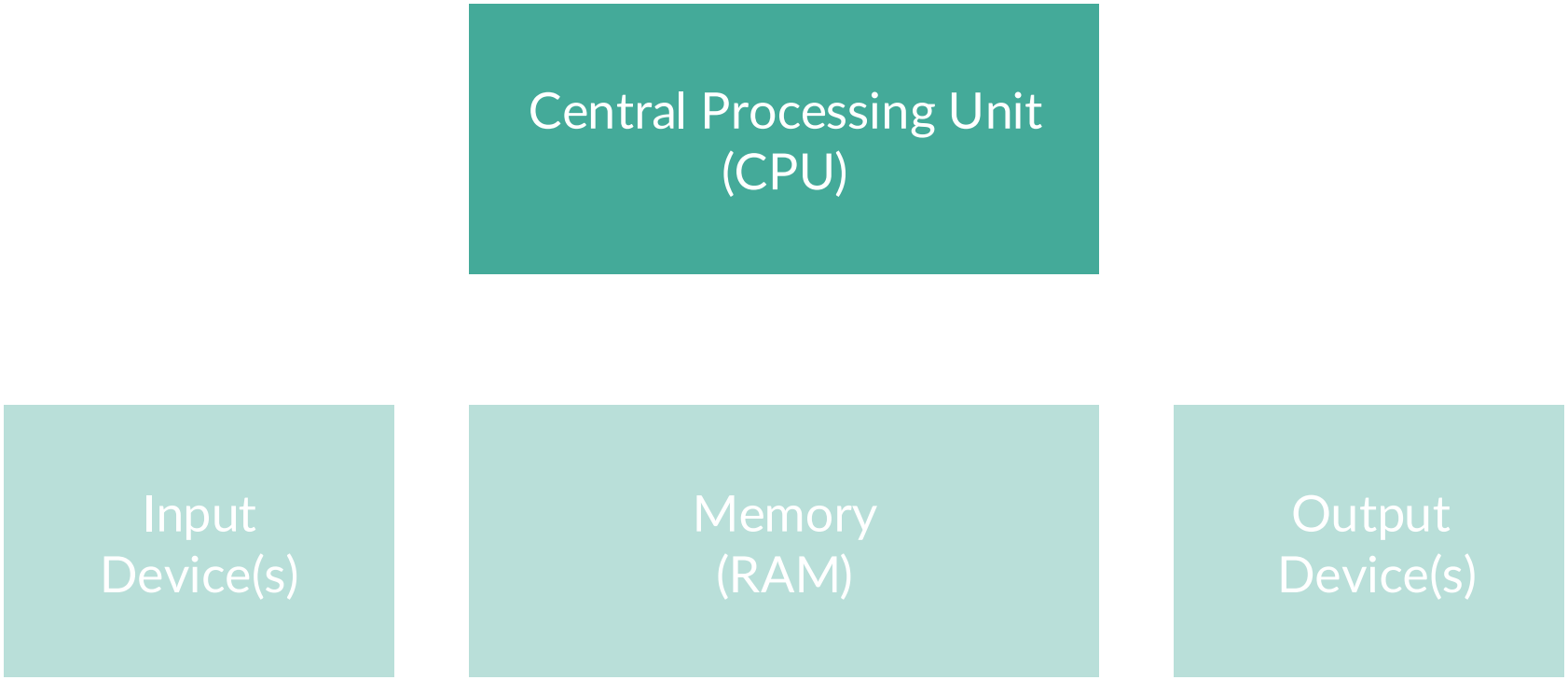
Logical Structure of a Computer

A computer consists of four main components:



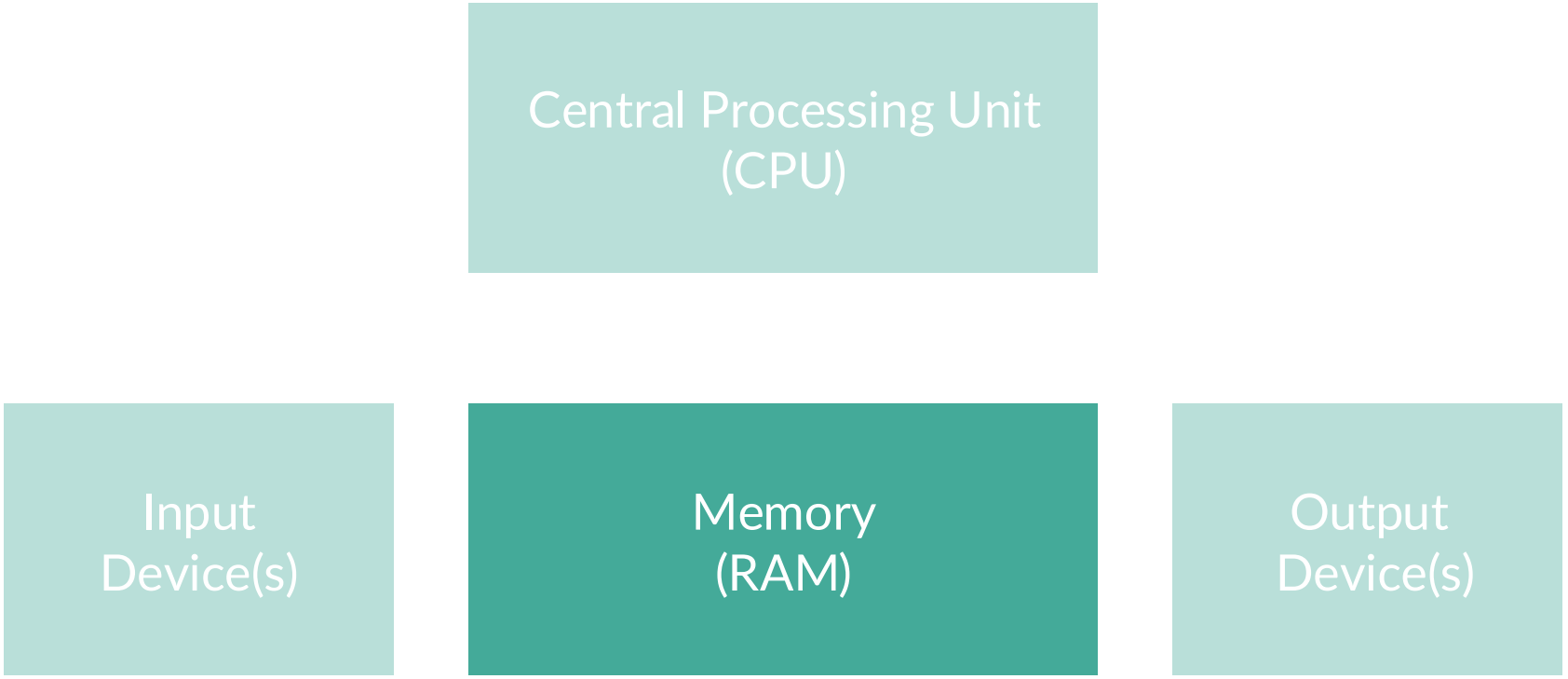
Central Processing Unit

The CPU fetches the next instruction in a program from memory and executes it.



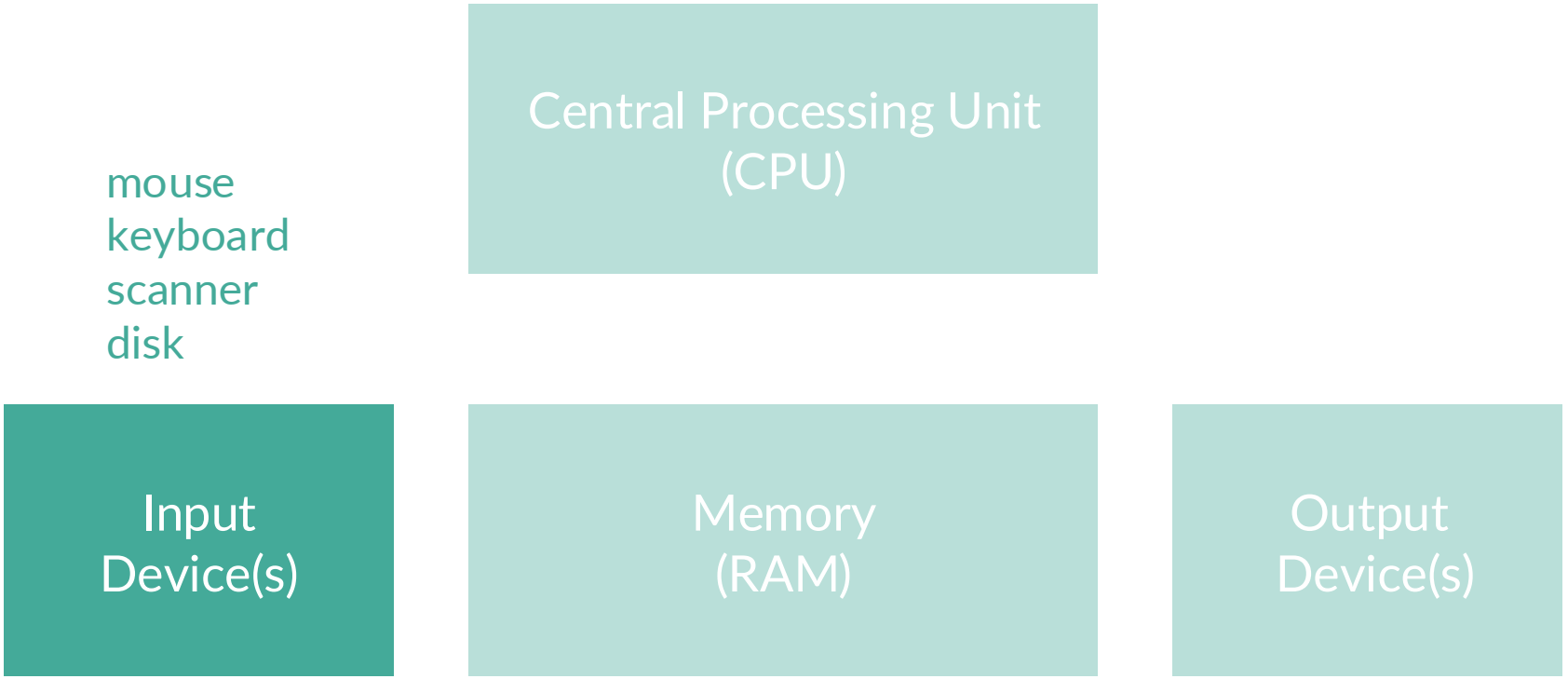
Memory

Memory is a storage array that stores (1) computer program instructions and (2) data used by the program.



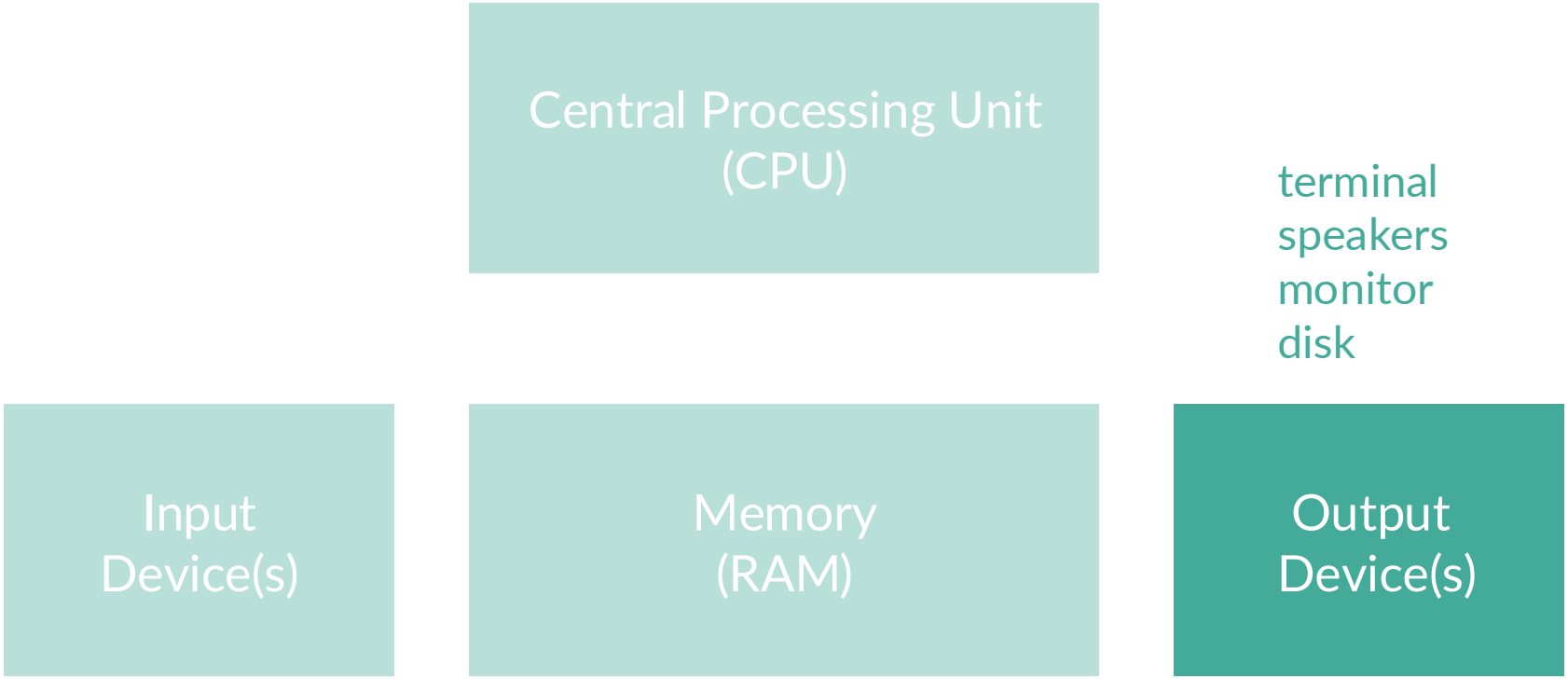
Input Devices

Input devices provide input data into the computer.

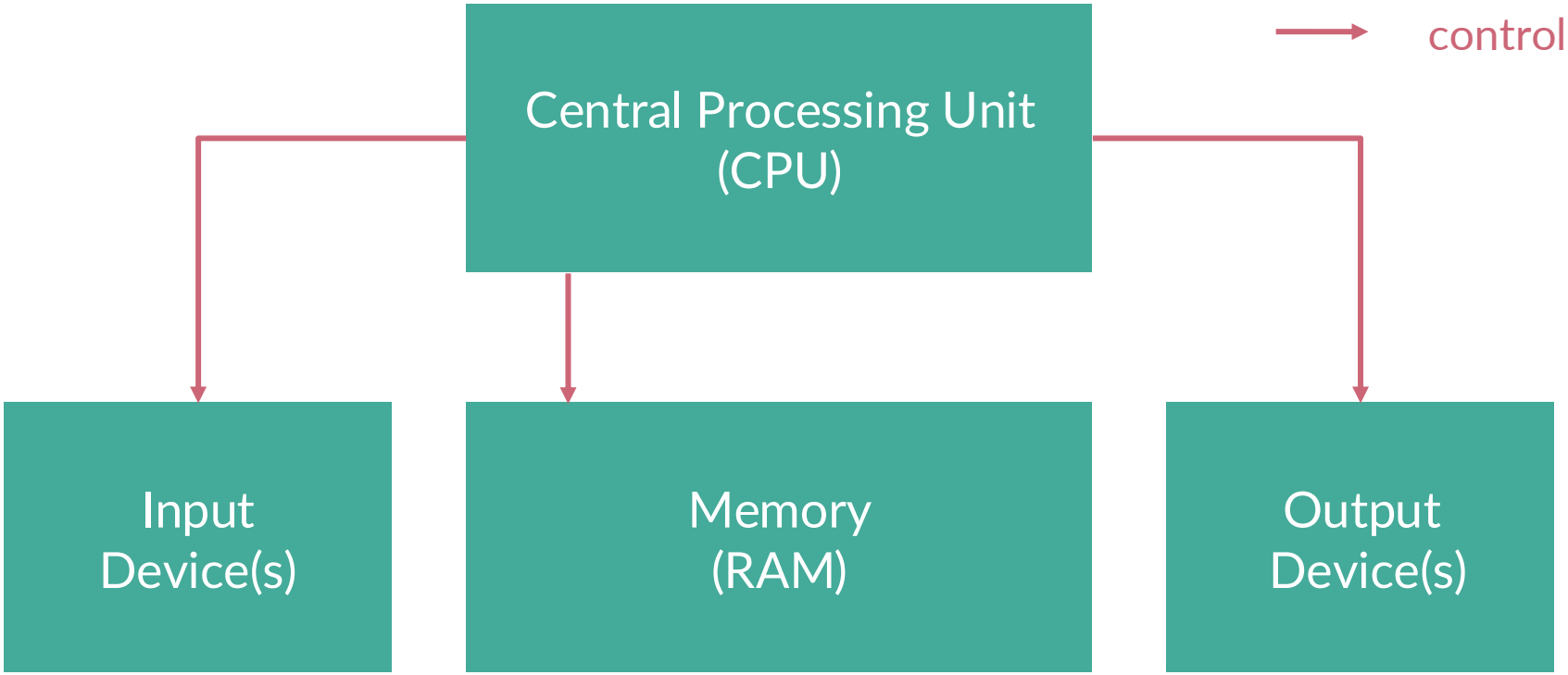


Output Devices

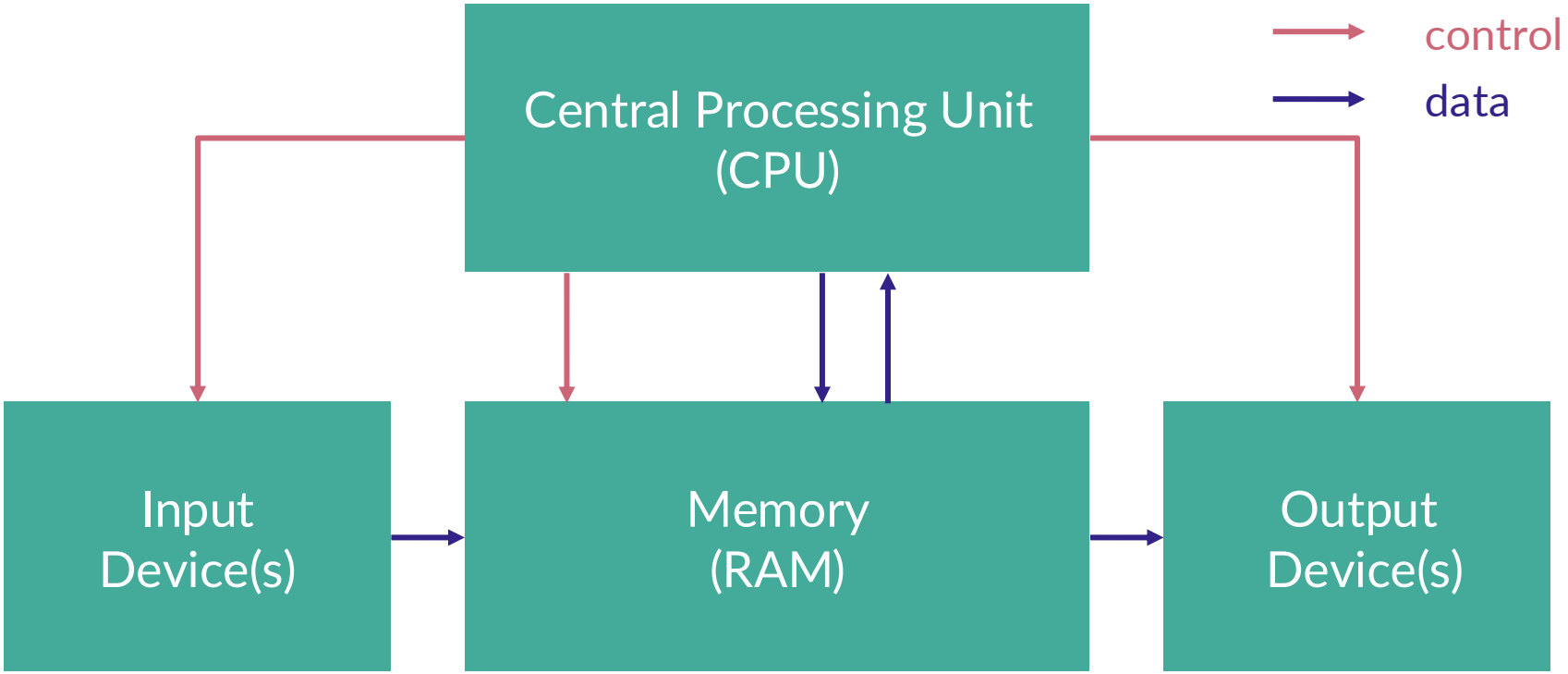
Output devices allow the computer export data.



The CPU Provides Control

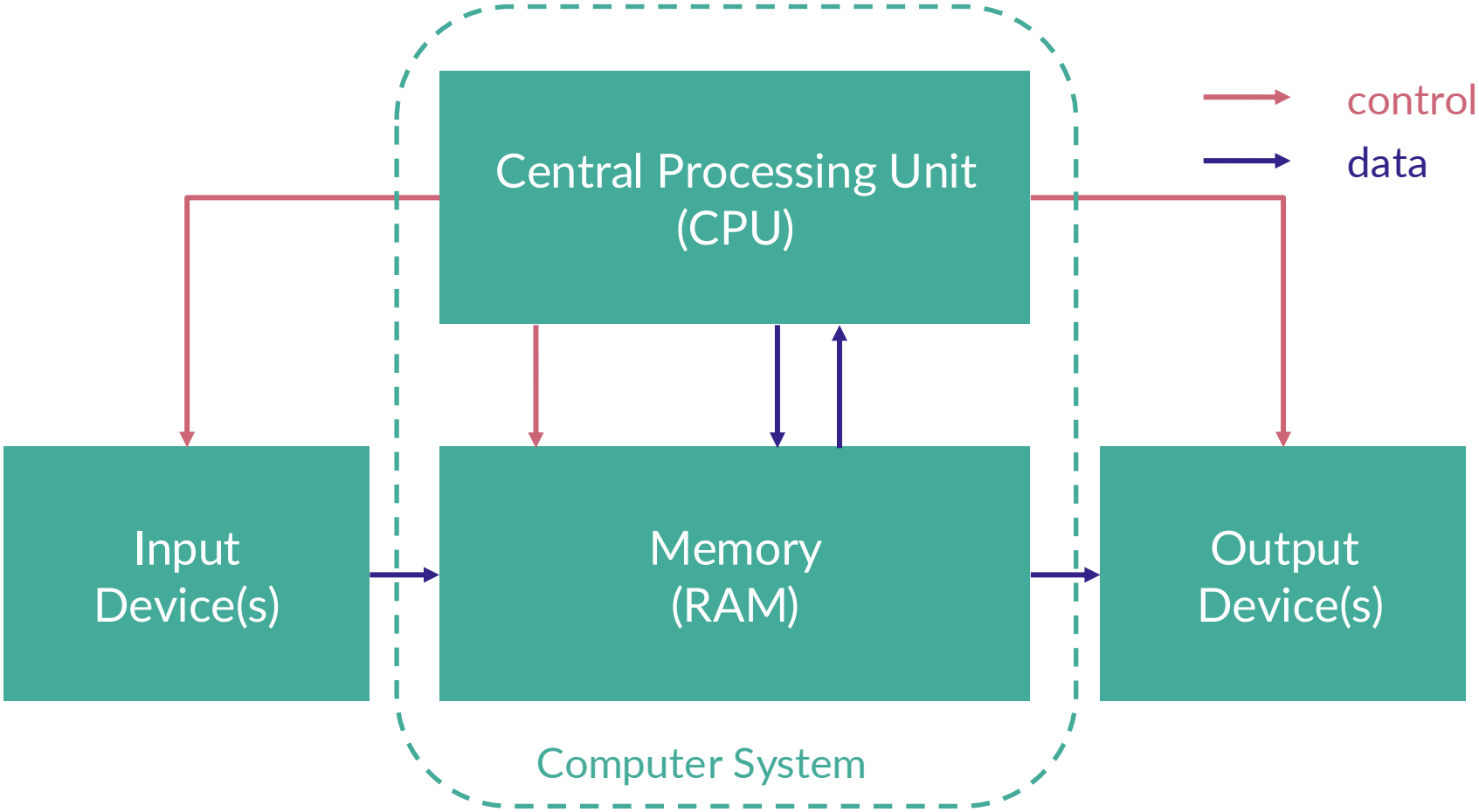


All Data Goes Through Memory



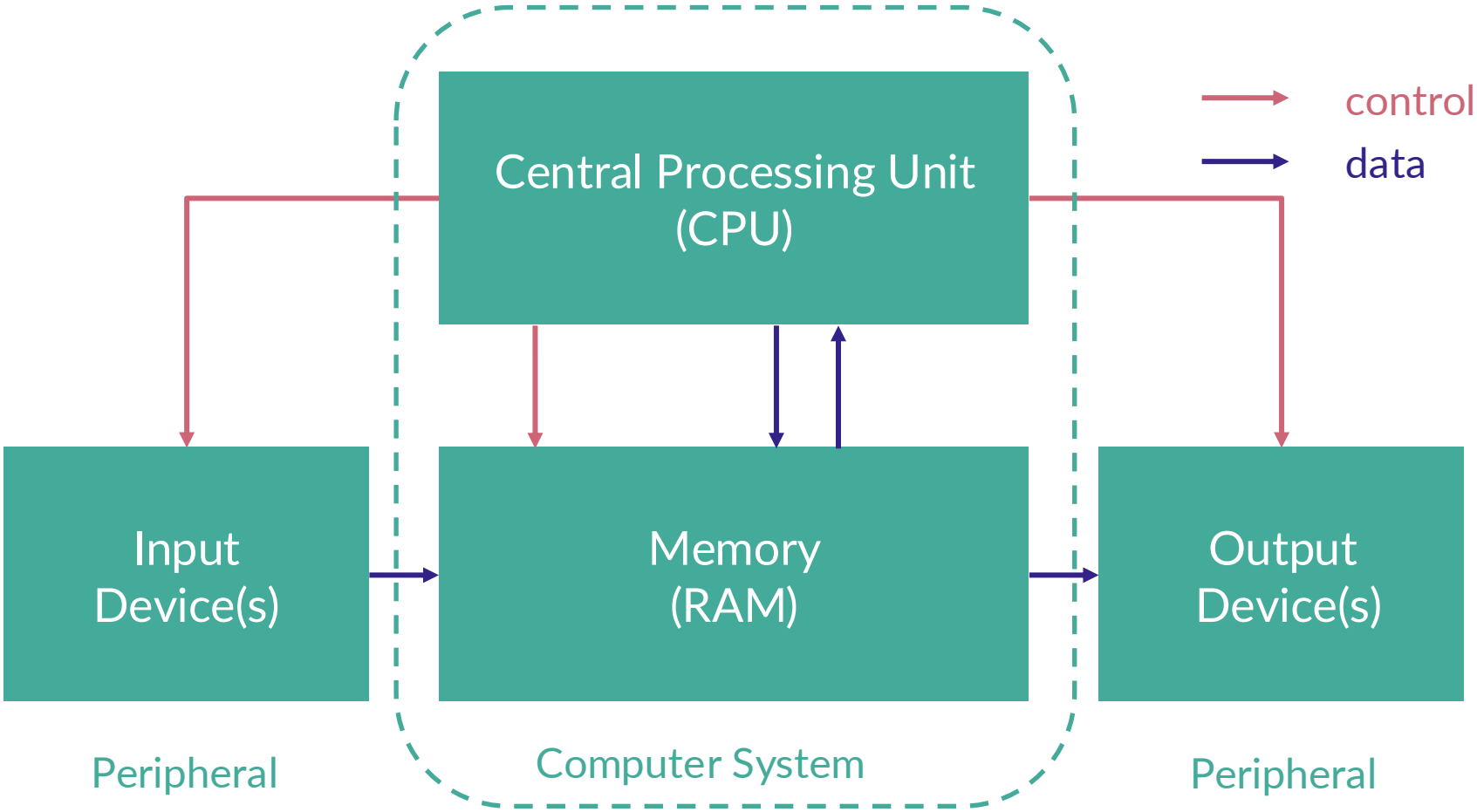
Computer System

The computer system performs computation.



Peripheral Devices

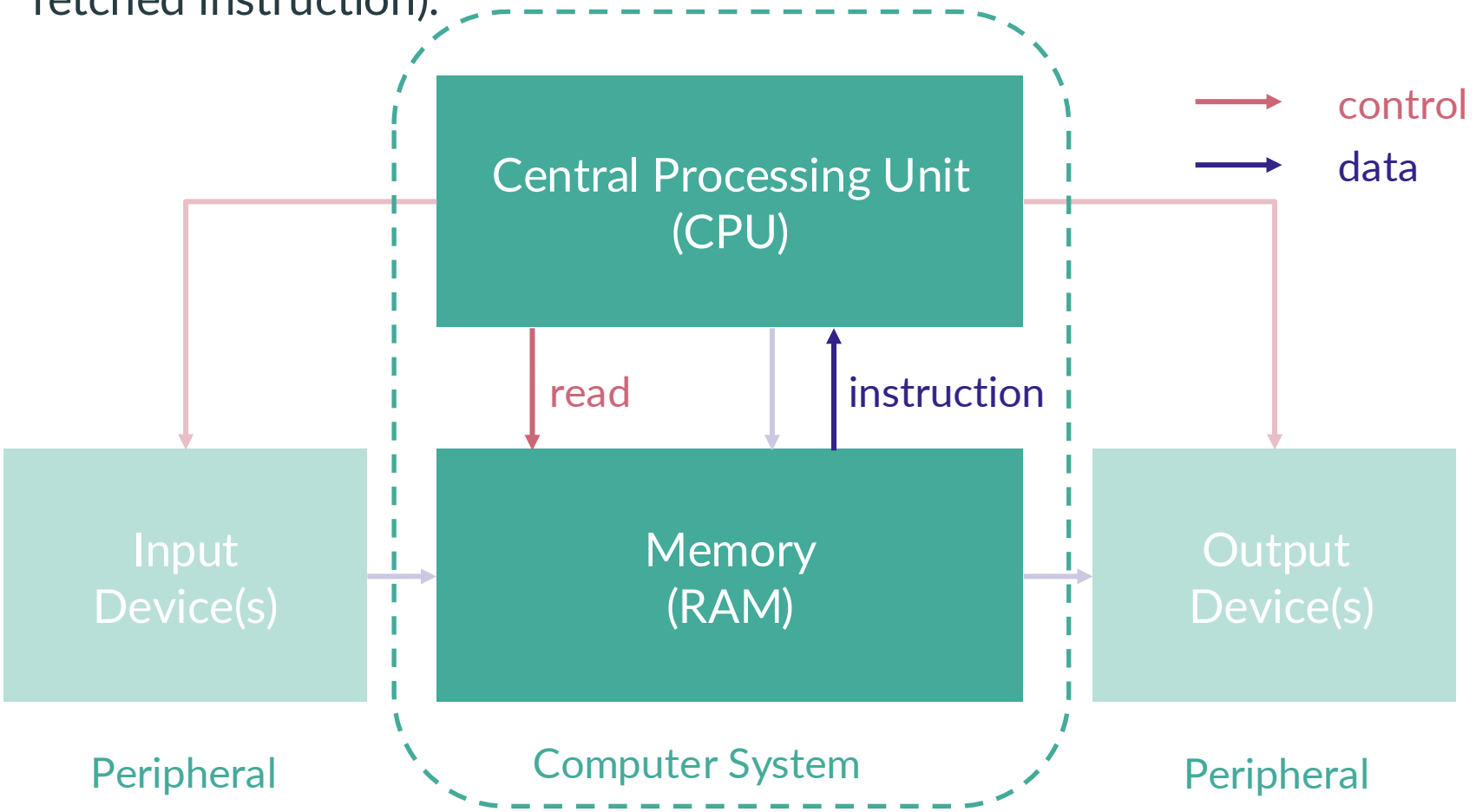
Peripheral devices provide inputs and outputs, but not computation.



Computer System

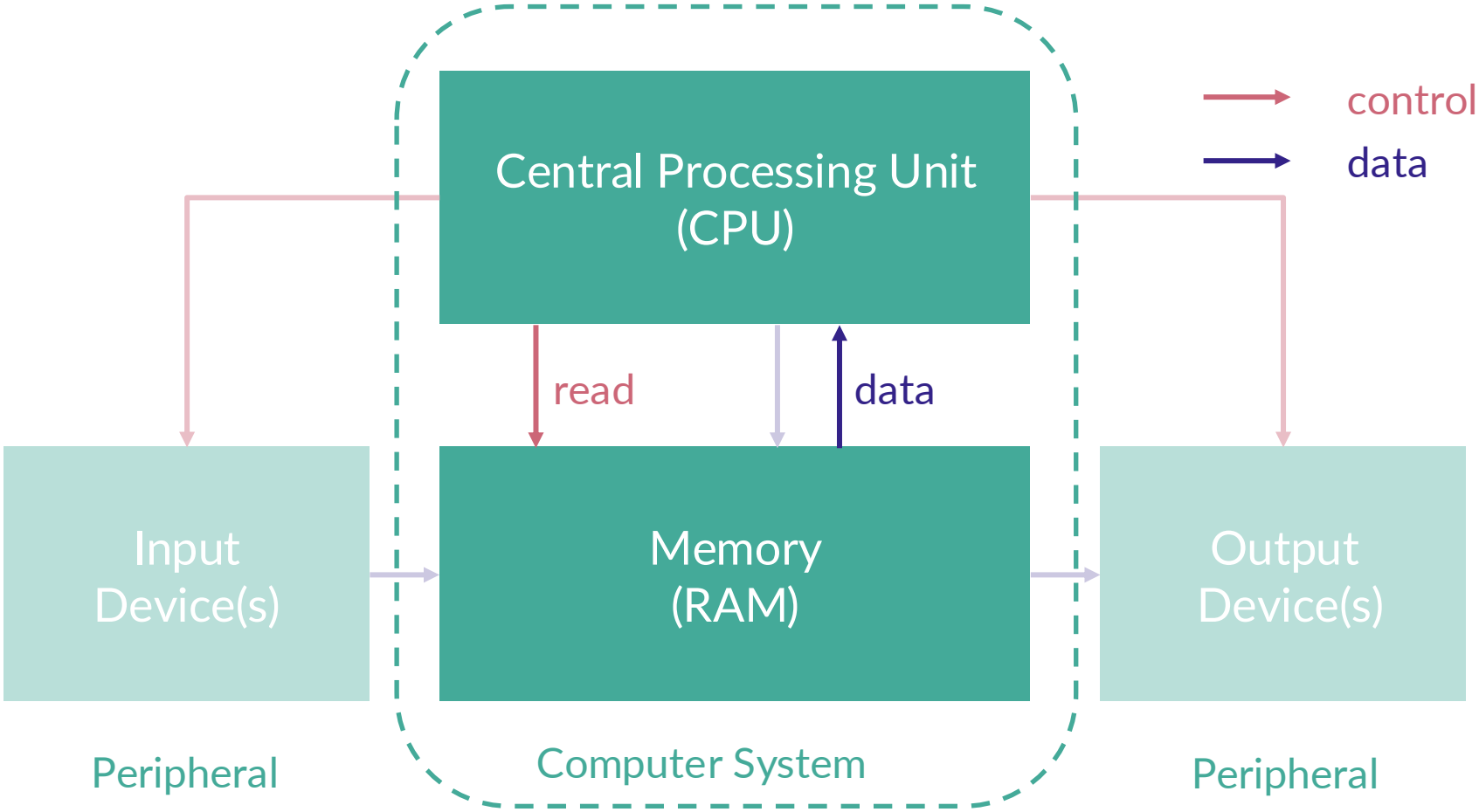
The CPU Reads Instructions

The CPU can read the next instruction from memory (then execute the fetched instruction).



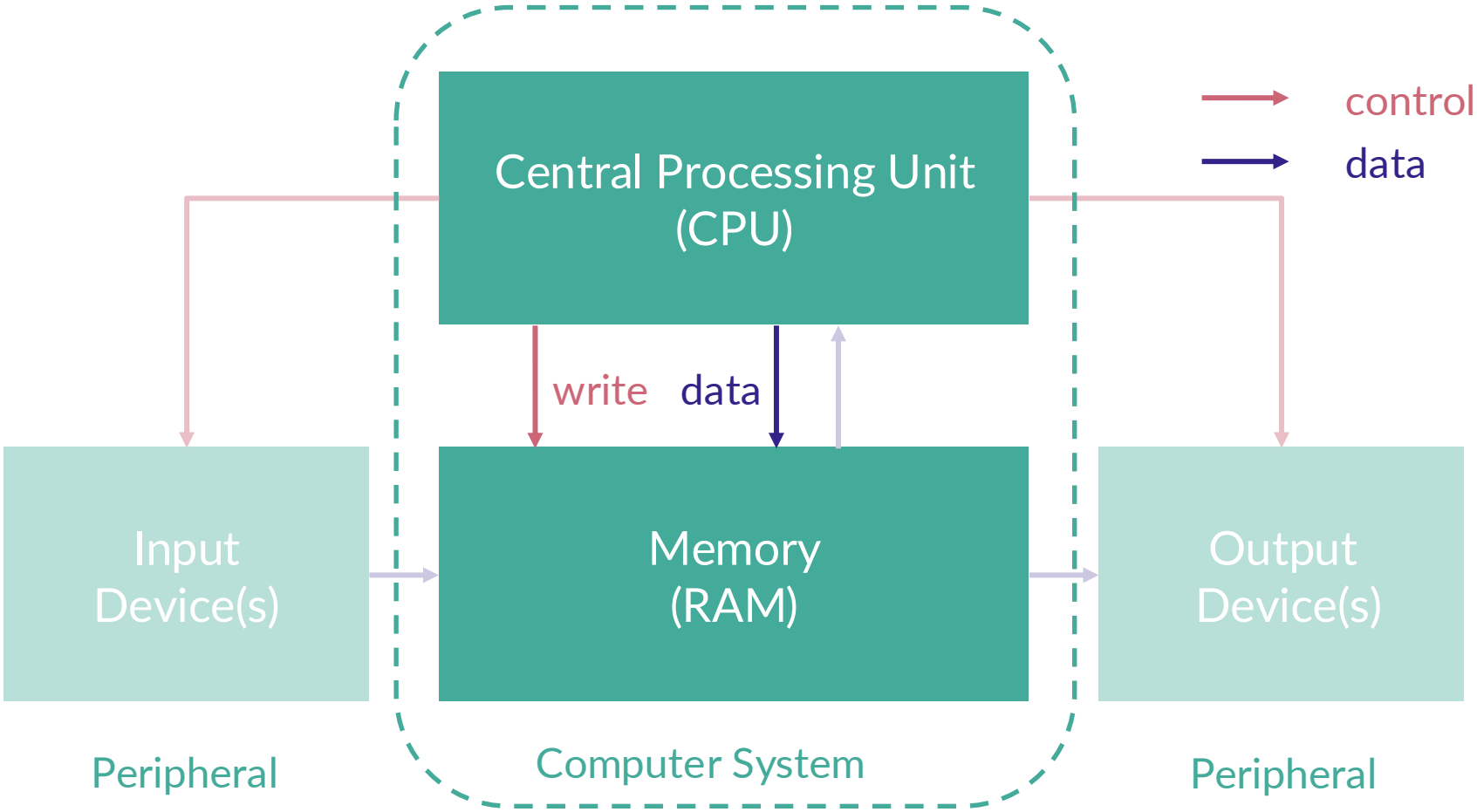
The CPU Also Reads Data

The CPU can read data from memory which is used by the instruction.



The CPU Writes Data

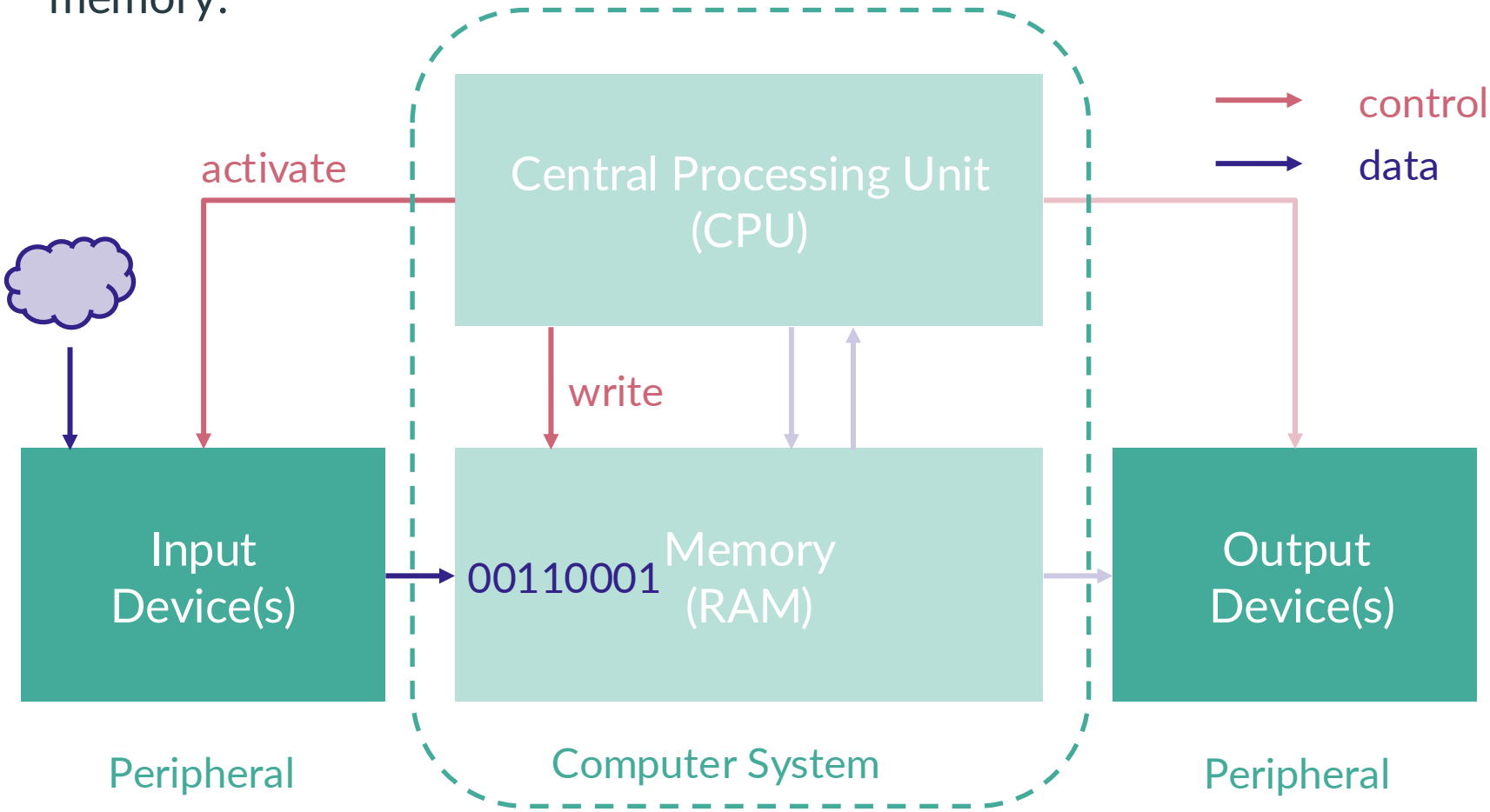
The CPU can write the result of computation to memory.



Peripheral Devices

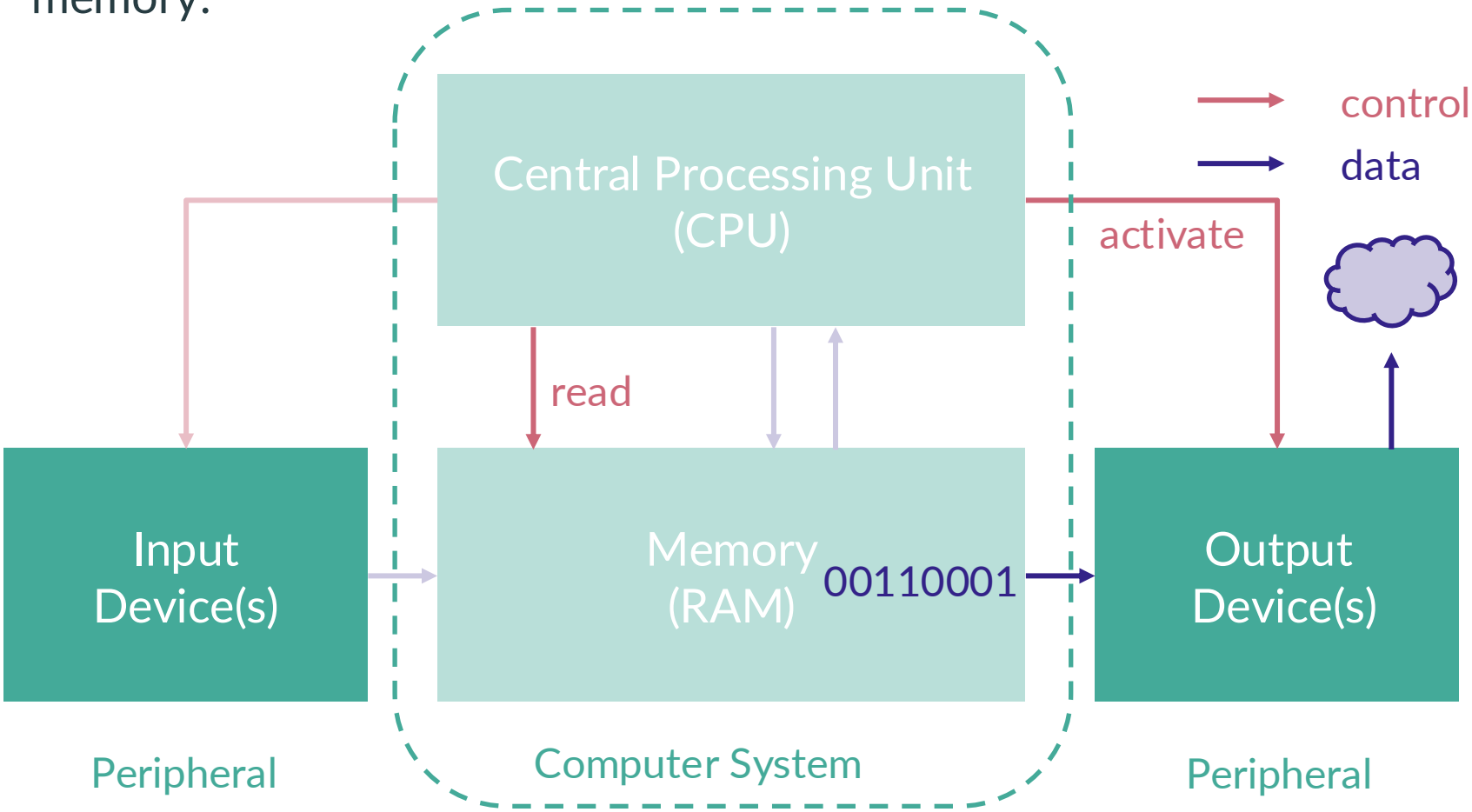
Input Devices Read Data

The CPU instructs the input device to read user input and store it in memory.



Output Devices Display Data

The CPU instructs the output device to display data that is stored in memory.



Peripheral Devices are Translators

Input devices translate human-friendly data representations into machine-friendly data representation.

‘A’ -> 01000001

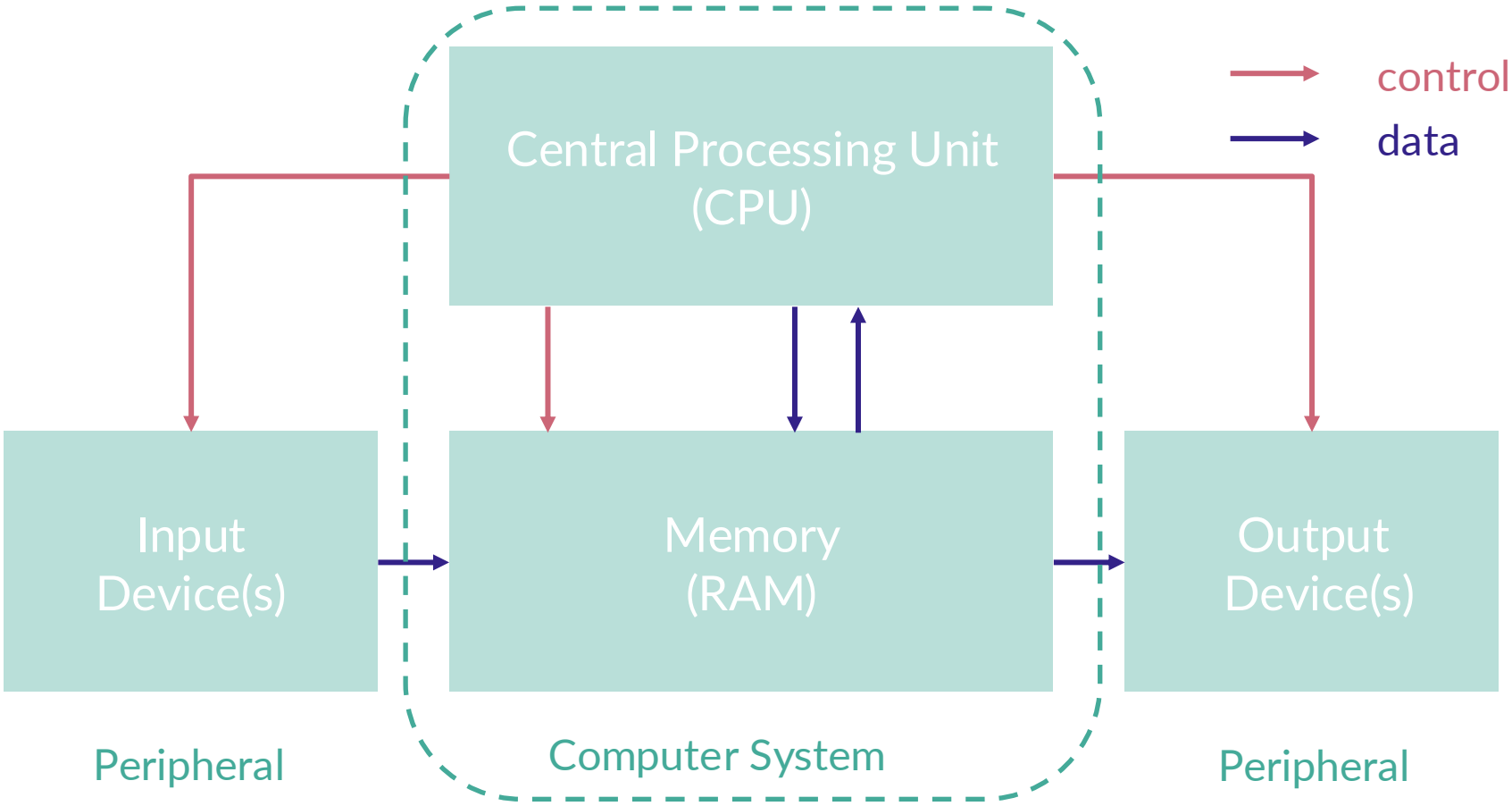
Output devices translate machine-friendly data representation into human-friendly data representation.

01000001 -> ‘A’

Interconnect

Components Need to Communicate

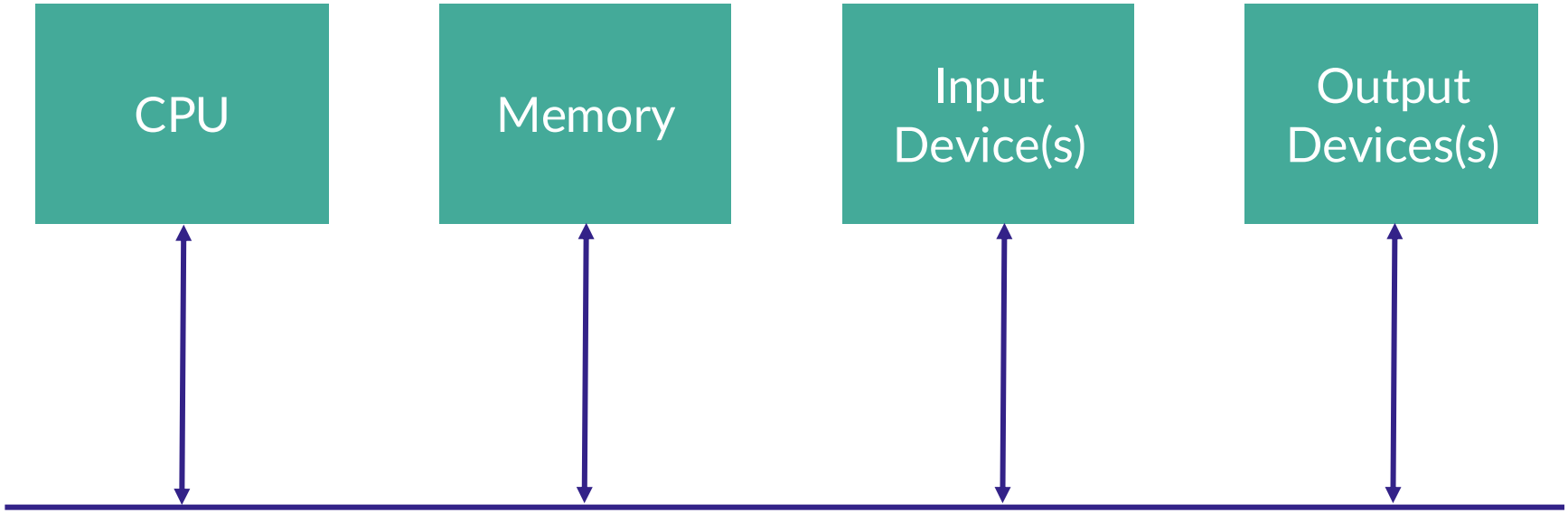
These communication channels are *logical*, there are many different ways to implement the connections.



Communication Via a Shared Bus

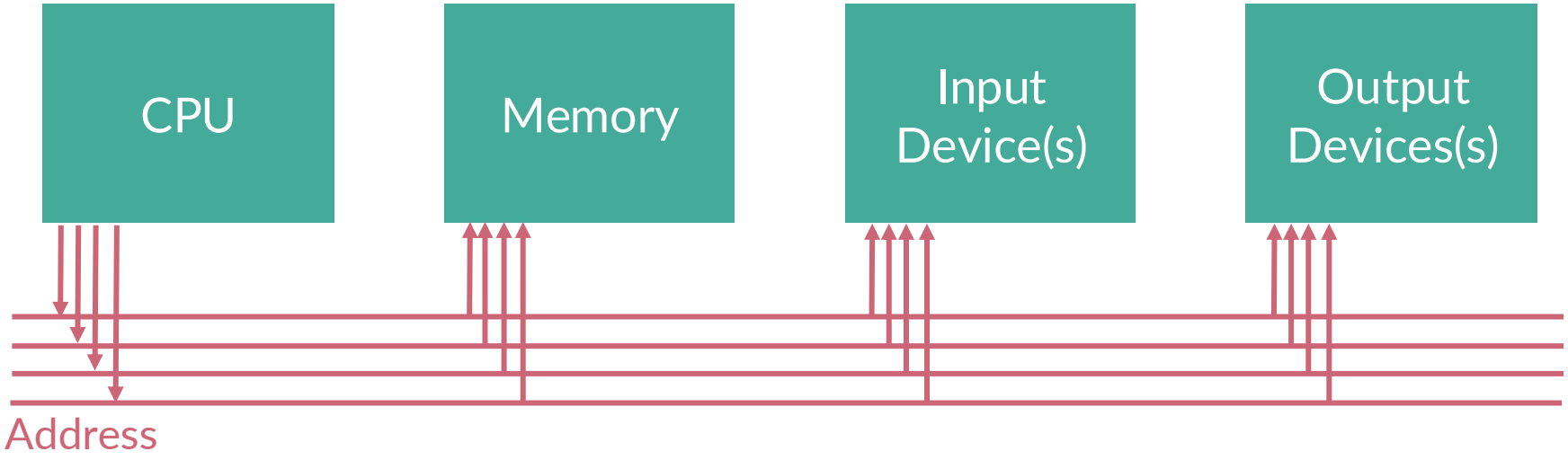
A **bus** is a set of wires used to transmit information between components.

A **shared bus** is a bus that connects multiple components using the same set of wires.



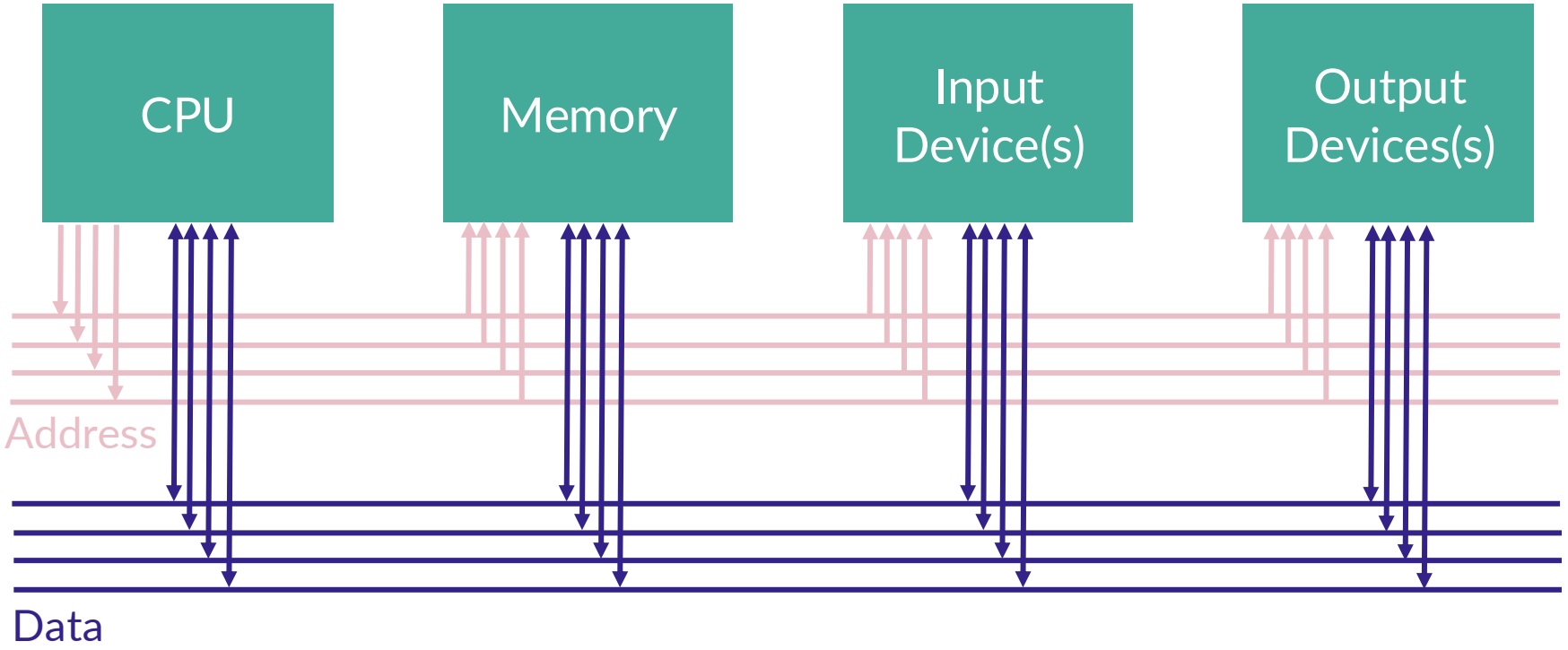
Address Bus

The address bus transmits an address in memory (used sometimes as an identifier).



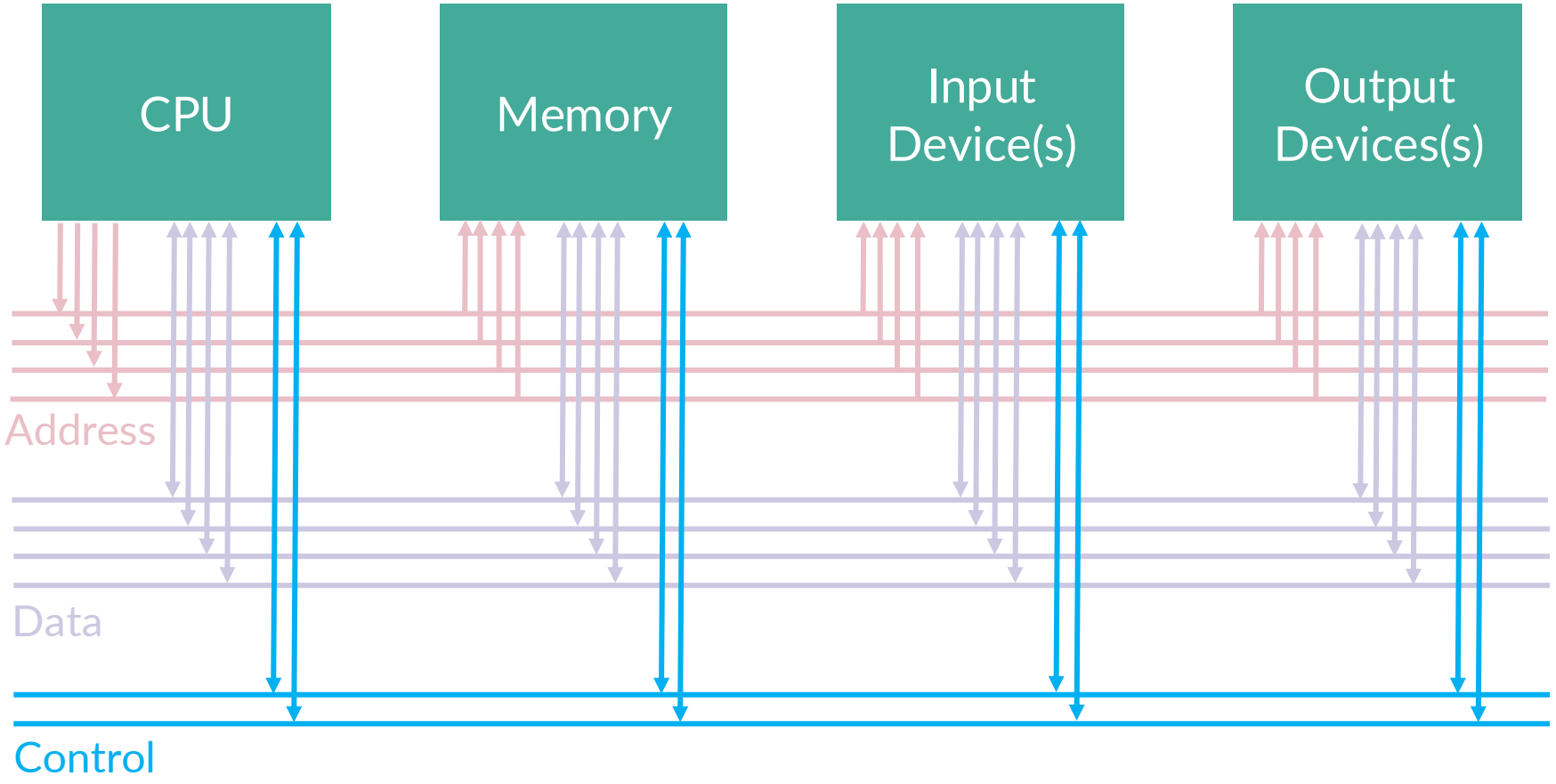
Data Bus

The **data bus** transmits data.



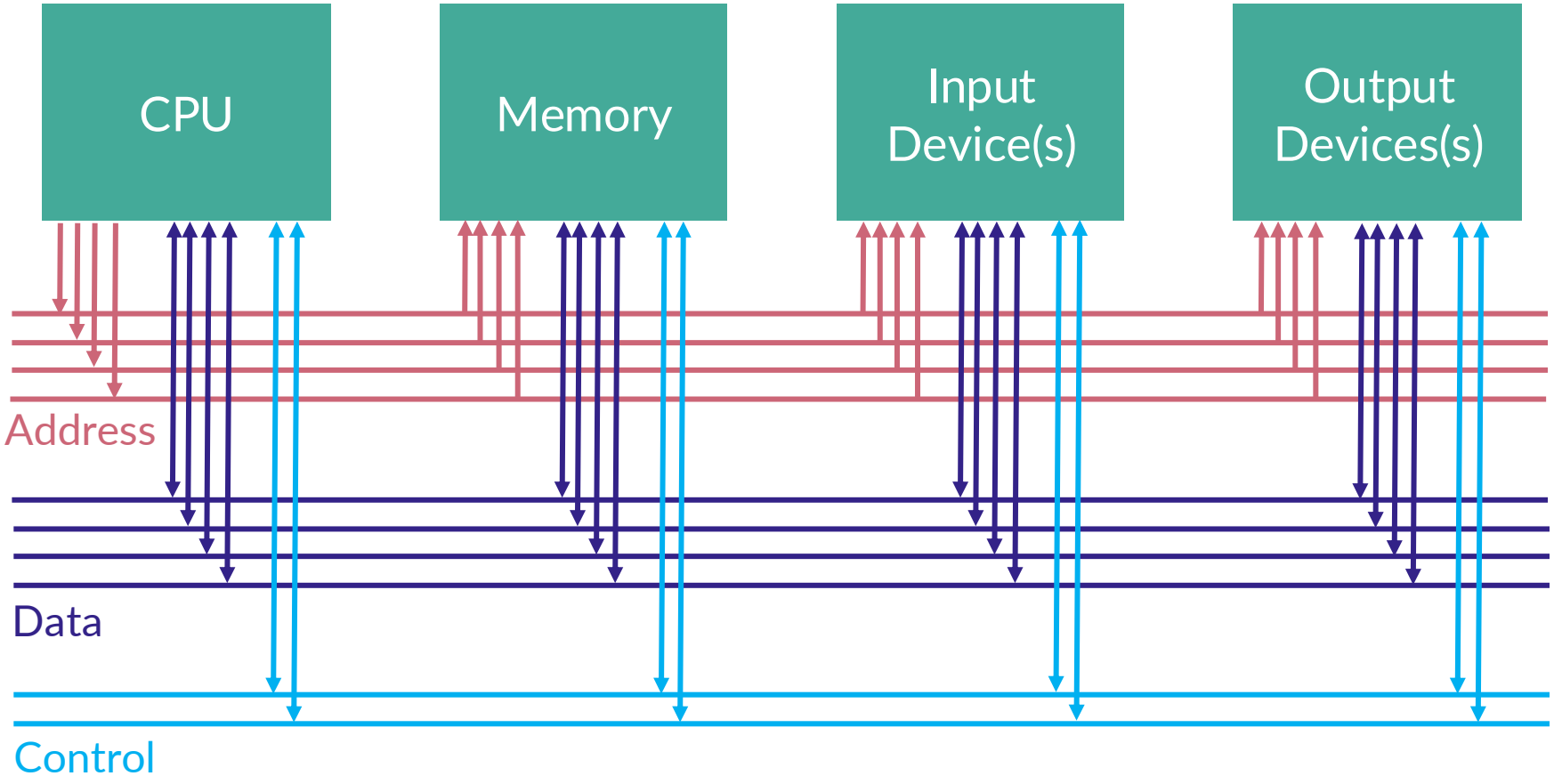
Control Bus

The **control bus** transmits command information.



System Bus

The **system bus** consists of the address, data, and control busses.



Communication Over The Bus

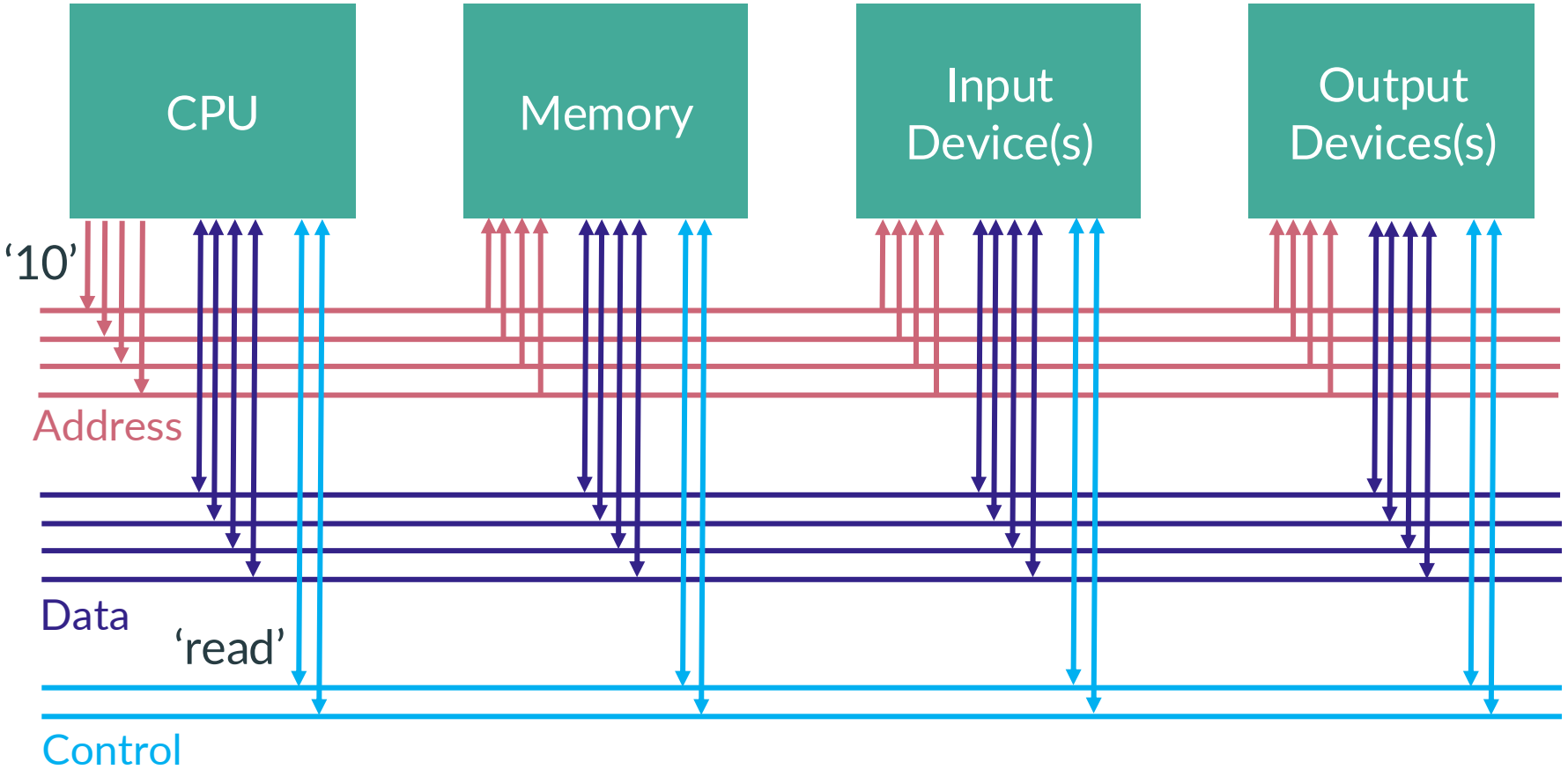
A **bus protocol** is a set of rules that define how components communicate over a shared interconnect.

Bus protocols:

- define how data transfer is initiated and completed
- specify the meaning of control and data signals
- coordinate roles and responsibilities between components
- provide a standardized interface

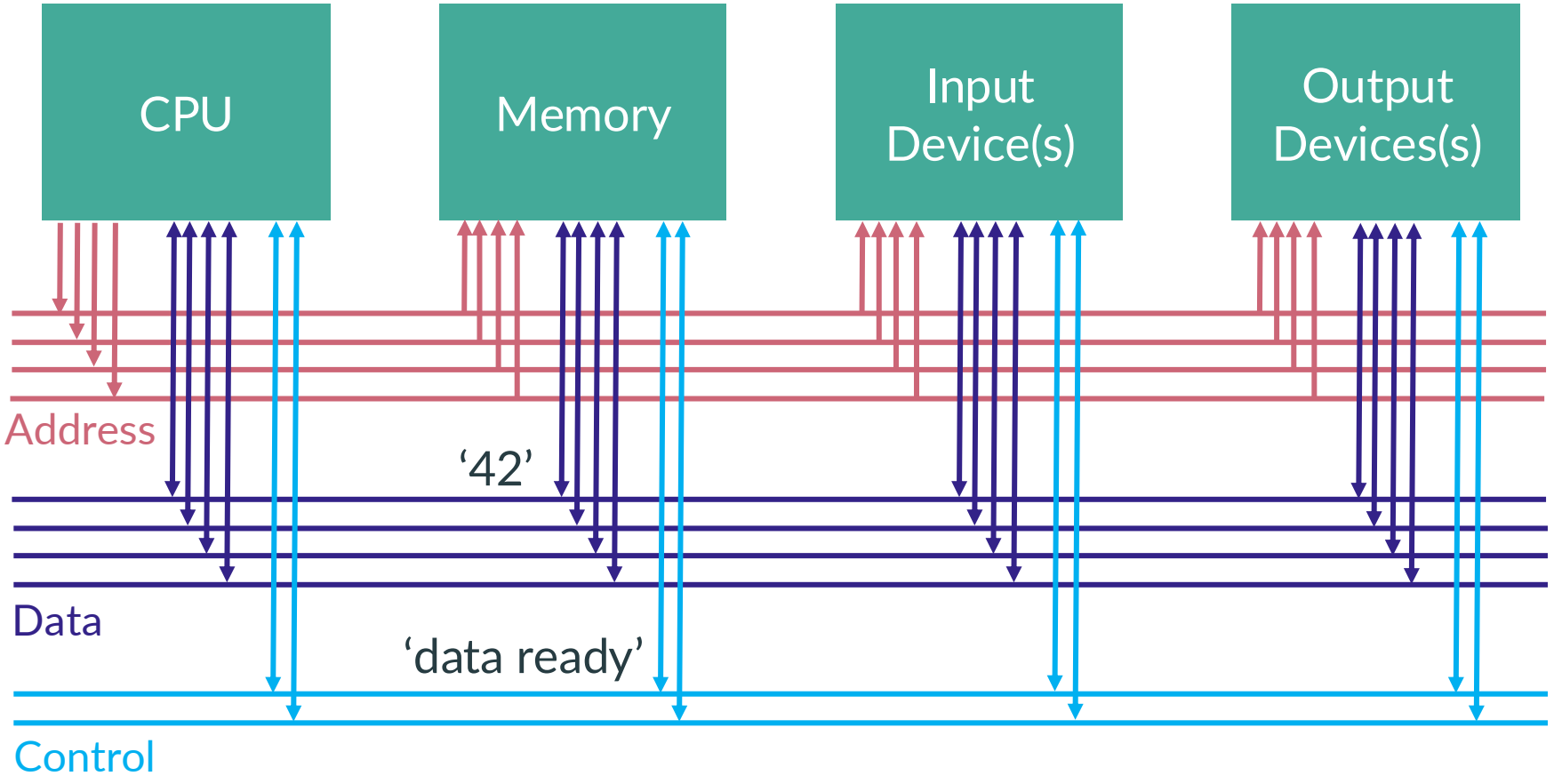
CPU Read Via the System Bus

First, the CPU (1) sets address bus to read address and (2) sets control bus to ‘read’



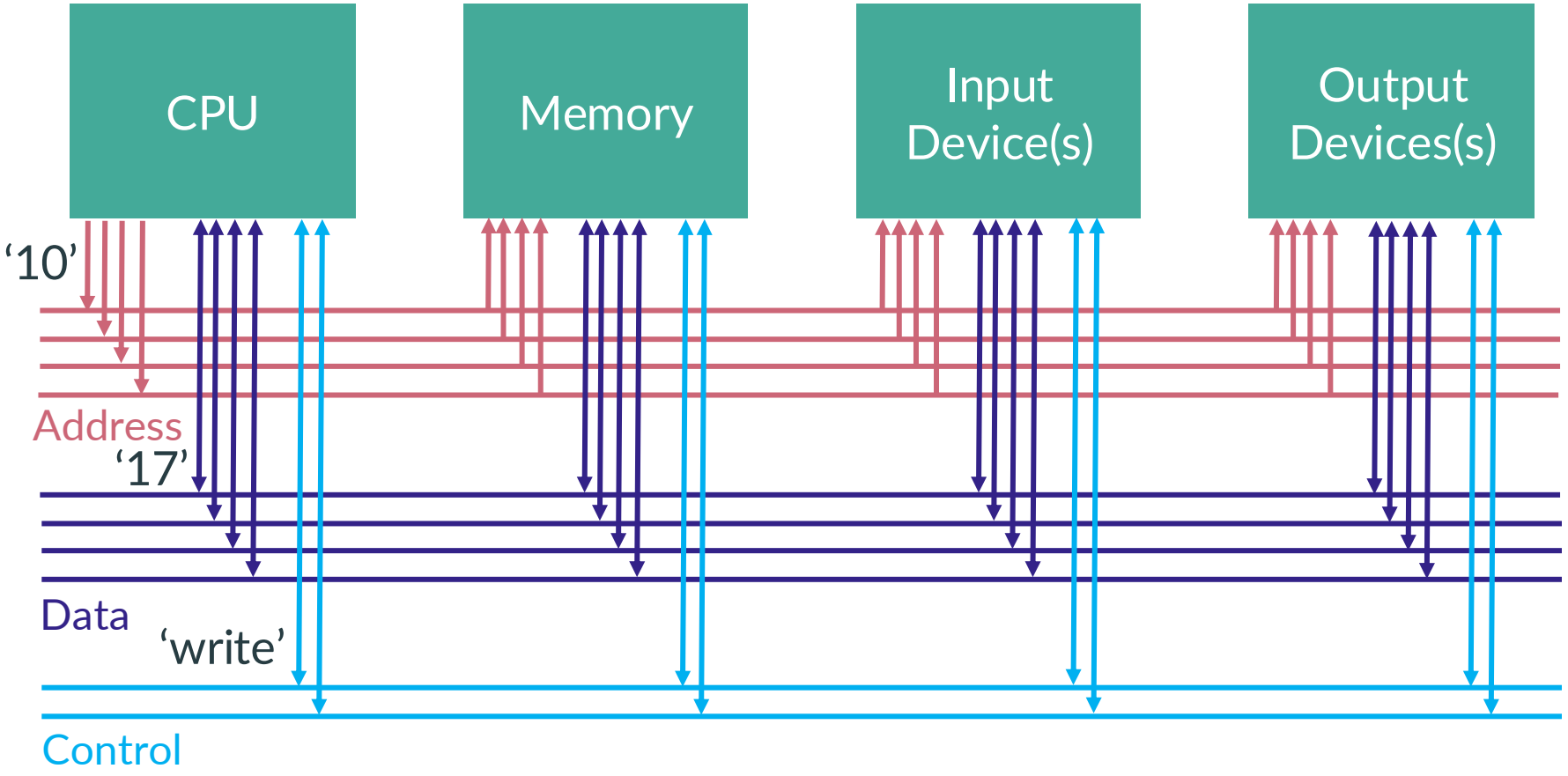
CPU Read Via the System Bus

Then, the memory (1) sets data bus to bits at specified address and (2) sets control to ‘data ready’



CPU Write Via the System Bus

First, the CPU (1) sets address bus to write address, (2) sets the data bus to write data, and (3) sets control bus to 'write'



CPU Write Via the System Bus

Then, the memory (1) sets the control bus to ‘acknowledge’ after writing the data at the specified location

